SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	100 Marks (4 × 25)
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Prabhakaran	**Reg. No.:**	1912260199
Section / Batch:	Batch 1	**Date:**	20.10.2026

Code of Conduct

I, Prabhakaran, 1912260199 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q5**

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Scenario Based Assignment

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

A cost-aware, frequently updated A* planner offers a strong balance of route quality, safety, and responsiveness. Its reliability depends on accurate maps, sensible cost penalties, and continuous re-planning when conditions change.

Conclusion

A* is the best baseline when the cost model includes travel time and safety risk, and when optimality matters. For strict real-time operation, use an incremental or repeated A* variant with current sensor updates, a practical time limit, and a safe fallback route. Greedy search may be useful for a quick initial estimate, but should not be the sole decision rule for life-critical deliveries.

(e) Recommendation

When new information reports a blocked path, the drone must mark the edge or area unavailable, update its map and costs, and re-plan. A previously selected route may become invalid, so the system needs frequent updates and fast re-planning. Reusing unaffected search information can reduce repeated work. Sensor uncertainty should be handled conservatively by assigning risk penalties or avoiding poorly verified routes.

(d) Dynamic blocked paths

A good heuristic estimates remaining cost without systematically overestimating it when optimality is required. Straight-line distance is often a useful lower bound, but it may be misleading when rivers, damaged bridges, strong winds, or restricted flight corridors force detours. Greedy search is especially sensitive because it relies almost entirely on h(n). A* is less dependent on the heuristic because it also considers g(n), although a poor heuristic can still increase the number of nodes explored.

(c) Heuristic accuracy

A* evaluates each node using f(n) = g(n) + h(n). Here, g(n) is the actual accumulated travel cost, including time, terrain difficulty, and risk penalties, while h(n) estimates the remaining cost. A* balances progress already made with estimated distance remaining. With a suitable admissible heuristic and appropriate graph assumptions, A* can find an optimal least-cost route. For emergency delivery, the cost model should represent delay and safety, not distance alone.

(b) A* Search

Greedy Best-First Search selects the frontier node with the smallest heuristic value h(n), such as straight-line distance to the delivery point. It prioritizes apparent closeness to the goal and can find a route quickly when the heuristic is useful. However, it ignores the cost already travelled, g(n). A route that looks short may contain flooded ground, severe weather, or a blocked passage. Therefore, it is not guaranteed to find the lowest-cost or safest route.

(a) Greedy Best-First Search

The drone route-planning problem is an informed-search problem because the drone has a destination, different movement costs, and a heuristic estimate of remaining distance. Its route must minimize delay while avoiding blocked or unsafe paths.

Introduction

ANSWER – QUESTION 1 (25 MARKS): EMERGENCY MEDICINE DELIVERY DRONES

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption.
- Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

A hybrid global-local optimization strategy is more scalable and adaptable than exhaustive search or Hill Climbing alone. Real-time measurements and safety constraints are essential because the best timing plan changes with traffic demand.

Conclusion

Use a hybrid approach: a Genetic Algorithm or Simulated Annealing to generate improved coordinated plans, followed by fast local adjustments as new traffic measurements arrive. A multi-objective or weighted score should balance waiting time, emissions/fuel use, queue length, pedestrian delay, and fairness between roads. Apply hard safety constraints, test plans in simulation, deploy gradually, and retain a safe fallback timing plan.

(e) Recommended approach

A Genetic Algorithm encodes a complete or partial timing plan as a chromosome. It evaluates a population of plans, selects better candidates, and applies crossover and mutation to create new plans. The population explores several areas of the search space at once and can optimize interacting signals together. It may require substantial computation and can converge prematurely, so it needs diversity, feasible-solution handling, and a fixed run-time budget.

(d) Genetic Algorithms

Simulated Annealing sometimes accepts a worse neighbouring solution to escape a local optimum. Early in the search, a higher temperature allows more exploratory moves; as temperature decreases, the system becomes more selective. This can discover better signal plans than strict Hill Climbing, but its quality depends on the cooling schedule and evaluation budget. It is useful for periodic optimization, provided every candidate remains within traffic-safety constraints.

(c) Simulated Annealing

A local optimum occurs when nearby changes do not improve the current plan even though a better plan exists elsewhere. A plateau occurs when many timing plans have similar scores, so the algorithm has no clear direction. A ridge occurs when

improvement requires a coordinated sequence of changes across several intersections; changing one signal alone may appear worse. These problems are realistic because traffic lights interact: an adjustment that helps one junction may send a queue to the next.

(b) Local maxima, plateaus, and ridges

Hill Climbing begins with a feasible signal plan, evaluates its objective score, and repeatedly moves to a neighbouring plan that improves the score. For example, it may lengthen a green phase on a congested road or adjust an offset to reduce a queue. It is simple, fast, and can work well for small local adjustments. However, it accepts only improvements and may stop before reaching a globally good solution.

(a) Hill Climbing

Represent signal timing as an optimization problem. Decision variables may include green-light duration, phase order, and offsets between intersections. The objective is to minimize a weighted total of vehicle waiting time, fuel use, queue length, and congestion, subject to minimum/maximum green times, pedestrian crossing needs, safety rules, and coordination constraints. Traditional exhaustive search is inefficient because the number of timing combinations grows rapidly across hundreds of intersections and traffic changes continuously.

Introduction and formulation

ANSWER – QUESTION 2 (25 MARKS): SMART CITY TRAFFIC SIGNAL OPTIMIZATION

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Online search is suitable because it combines movement with learning. Local heuristic guidance makes exploration efficient, while frequent sensing, uncertainty-aware costs, and safety checks reduce the danger of navigating unknown terrain.

Conclusion

Use a risk-aware online agent with local heuristic planning, incremental map updates, and repeated re-planning. It should prioritize safe traversability and energy budget alongside distance to the scientific target. Use conservative obstacle margins, check sensor confidence, and select a safe fallback or return route if uncertainty becomes too high. Periodic planning prevents one outdated route from controlling the entire mission.

(e) Recommended approach

Uninformed methods such as breadth-first search explore without a goal-distance estimate and can waste time in large spaces. Informed methods such as A* use a heuristic to guide movement toward a target, but ordinarily rely on a sufficiently known search graph. Online search emphasizes acting and learning in the environment, rather than requiring the full map beforehand. An online rover can still use heuristic guidance, such as A* on its currently known local map, and re-plan as new data arrives.

(d) Comparison with uninformed and informed search

The rover begins with a map of known terrain and unknown cells. It combines camera, range, wheel, and other sensor readings to update its local map and estimate its current position. Each attempted move provides feedback about travel cost and traversability. Newly detected rocks or craters are added as obstacles; uncertain regions receive risk costs. The planner then recalculates a safe route to a sample location or chooses an information-gathering action.

(c) Internal model updates

Sensors may not reveal a hazard hidden behind a rock, and measurements can be noisy or incomplete. The rover may not know whether a surface is stable, how steep a slope is, or whether a passage is traversable until it approaches. Communication delay and limited energy also constrain decisions. The rover should maintain uncertainty estimates, preserve a safe stopping distance, and avoid actions whose possible outcomes exceed its risk limits.

(b) Partial observability and uncertainty

Offline search assumes the environment and state-transition information are sufficiently known before the plan is produced. On Mars, the rover has limited sensors and discovers terrain as it travels. An online agent can choose an action using current information, observe the result, and plan again. This supports exploration where no complete route is initially available.

(a) Why online search is required

The rover operates in an unfamiliar environment and cannot obtain a complete map before moving. It must alternate between sensing, updating its knowledge, planning a short action, and moving safely. This is the setting for an online search agent.

Introduction

ANSWER – QUESTION 3 (25 MARKS): AUTONOMOUS ROVER EXPLORING MARS

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

CSP modelling makes timetable rules explicit and verifiable. Backtracking finds a consistent assignment, while forward checking and informed variable ordering improve efficiency as the number of courses grows.

Conclusion

Use a CSP solver with backtracking, forward checking, and variable-ordering heuristics. A constraint graph helps identify exams that compete for students or rooms. If optimizing preferences, use branch-and-bound or a local improvement stage after finding a feasible schedule. Keep manual review for exceptional accommodation cases and publish only after automated conflict checks.

Recommended strategy

Large universities may have thousands of students, shared courses, limited specialist halls, part-time faculty, and last-minute changes. Constraints can interact, and incomplete enrolment data can cause conflicts. A scalable solver should use conflict graphs, domain filtering, minimum-remaining-values ordering, degree heuristics, and forward checking. Separate hard constraints from soft preferences, validate the final timetable, and allow controlled re-optimization when enrolment or availability changes.

(d) Real-world challenges and scalability

After assigning an exam to a slot, forward checking removes incompatible slots from the domains of unassigned exams. For example, the chosen slot is removed from the domain of another exam sharing students or a required hall. If any variable has an empty domain, the algorithm detects failure immediately and backtracks rather than exploring a doomed branch. Constraint propagation therefore reduces unnecessary search and helps reveal conflicts early.

(c) Constraint propagation and forward checking

Backtracking assigns a value to one exam variable at a time. It selects an available slot and hall, checks whether the assignment conflicts with already assigned exams, and continues if it is consistent. If no legal value remains, the algorithm

returns to an earlier variable and tries a different value. Choosing the most-constrained exam first (minimum remaining values) and selecting the variable involved in the most conflicts can reduce search effort.

(b) Backtracking search

Variables: each course exam, represented by a variable E_i. Domains: the possible date/time slots and eligible halls for each exam. Constraints: (1) exams attended by the same student cannot overlap; (2) room capacity must be at least the number of candidates and room facilities must be suitable; (3) only the available number of halls may be used in a slot; (4) faculty or invigilator availability must be respected; (5) prerequisite or specified subject-order rules must be met; and (6) approved accommodations, such as accessible rooms or extra time, must be provided. Soft constraints can minimize long gaps, consecutive exams, and inconvenient schedules.

(a) CSP formulation

Exam timetable generation is a Constraint Satisfaction Problem (CSP). The objective is to assign exams to time slots and halls while satisfying student, room, faculty, precedence, and accommodation requirements. A valid solution must satisfy all hard constraints; preferences such as avoiding late exams can be treated as soft constraints to optimize.

Introduction

ANSWER – QUESTION 4 (25 MARKS): UNIVERSITY EXAM TIMETABLE GENERATION

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.

(b) Analyze the computational challenges of large game trees.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

(d) Design an evaluation function and justify the factors considered.

(e) Recommend a strategy for balancing optimality and real-time performance.

ANSWER – QUESTION 5 (25 MARKS): STRATEGIC GAME AI FOR REAL-TIME DECISION MAKING

Introduction

The game AI must choose strong moves against a human player while making decisions within a strict time limit. Since each game state can lead to many possible moves, the decision tree becomes very large. Minimax, Alpha-Beta pruning, and an evaluation function can help the AI select a good move efficiently.

(a) Minimax algorithm

Minimax is an adversarial search algorithm used in two-player, turn-based games. It assumes that the AI tries to maximize its score and the opponent tries to minimize the AI's score. The AI builds a game tree containing possible moves and the resulting states. At the AI's turn, it selects the move with the highest value; at the opponent's turn, it assumes the opponent will choose the move with the lowest value for the AI. The values are passed back up the tree to select the best move. In a real-time strategy game, a full tree may be too large, so the AI usually searches to a limited depth and evaluates the resulting states.

(b) Computational challenges of large game trees

If each state has b possible moves and the search depth is d, Minimax may examine approximately O(b^d) nodes. The number of states therefore grows rapidly as the branching factor or search depth increases. This can cause high processing time and memory use, making the AI miss its response deadline. Real-time games also change quickly, and searching every possible move may be unnecessary. The AI must therefore limit the search, prioritize promising moves, and use efficient pruning.

(c) Alpha-Beta pruning

Alpha-Beta pruning improves Minimax by skipping branches that cannot change the final decision. Alpha (α) is the best value found so far for the maximizing player, while beta (β) is the best value found so far for the minimizing player. During the search, if α becomes greater than or equal to β, the remaining branches at that point can be pruned because they cannot produce a better choice for the player making the decision. For example, if the maximizing player already has a move worth 8 and another branch is shown to be limited to 5 or less, that branch does not need to be explored further. Alpha-Beta returns the same decision as Minimax when the same search depth and evaluation are used, but it can examine far fewer nodes. Good move ordering makes pruning more effective.

(d) Evaluation function and factors

When the search reaches its depth limit or the time budget is nearly over, an evaluation function estimates how favorable a game state is for the AI. One possible weighted function is:

Evaluation(state) = w (Resource advantage) + w (Unit strength) + w (Territory control) + w (Objective progress) − w (Threats and losses)

The weights (w to w) represent the importance of each factor and should be tuned and tested for the game. Resource advantage measures available resources; unit strength considers the number and effectiveness of units; territory control measures strategic areas held; objective progress measures progress toward winning; and threats and losses account for vulnerable units, damage, or risk. The function should reflect the actual game rules and be tested against different strategies so that it does not reward a short-term gain that leads to a worse overall position.

(e) Recommended strategy for real-time performance

Use depth-limited Minimax with Alpha-Beta pruning, good move ordering, and a carefully tested evaluation function. Set a strict computation budget and use iterative deepening: search depth 1, then depth 2, and continue while time remains. If the deadline arrives, return the best move from the last fully completed search. Search promising moves first, avoid evaluating clearly illegal moves, and reuse information from earlier searches where possible. For very large games, a transposition table can store evaluations of previously visited states. The AI should also be tested for decision quality and response time. This approach balances strong decisions with reliable real-time responses, although it may not guarantee the globally best move when the search is cut off.

Conclusion

Minimax provides a logical way to model competition between the AI and its opponent. Alpha-Beta pruning reduces unnecessary exploration without changing the Minimax result for the searched tree. A weighted evaluation function estimates complex states, while depth limits, iterative deepening, move ordering, and a strict time budget help the AI respond quickly. Together, these methods provide a practical balance between decision quality and real-time performance.